

Week 02 - Day 02 Home Assignments

Home Assignment:

1. Import the provided Postman collection.
2. Practice all **3 requests** available in the collection.
3. Take screenshots of the successful execution of each request.
4. Combine the screenshots into a **single PDF**.
5. Push the PDF to your **GitHub repository**.
6. Share the **GitHub repository link** in the **Assignment Tracker**.

Please complete the assignment before the next session.

Assignment 01: Text to Audio

The screenshot displays the Postman interface for a POST request to the endpoint `https://api.groq.com/openai/v1/audio/speech`. The request body is a JSON object with the following fields:

```
{
  "model": "canopylabs/orpheus-v1-english",
  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",
  "voice": "diana",
  "response_format": "wav"
}
```

The response status is **200 OK**, with a duration of 1.46 s and a size of 402.04 KB. The response is a WAV audio file, which is visualized in a player at the bottom of the interface. The player shows a progress bar at 0:08 / 0:08 and includes volume and settings icons.

Assignment 02: MultiLingual

Request Payload:

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [
    {
      "role": "system",
      "content": "You are an API Specialist. Always respond in the same language as the user."
    },
    {
      "role": "user",
      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லுங்கள்."
    }
  ]
}
```

Response:

```
{
  "id": "chatcmpl-d03e6bd2-9e4f-41b4-b54e-1ca0ea272506",
  "object": "chat.completion",
  "created": 1784694409,
  "model": "llama-3.3-70b-versatile",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "நன்றி! இந்த API-ஐ பயன்படுத்துவதற்கு, நீங்கள் பின்வரும் படிகளை பின்பற்ற வேண்டும்:\n\n1. **API பதிவு**: முதலில், நீங்கள் அந்த API சேவையை வழங்கும் இணையதளத்தில் பதிவு செய்ய வேண்டும். இதற்கு ஒரு பயனர் பெயர், மின்னஞ்சல் முகவரி மற்றும் கடவுச்சொல் தேவைப்படும்.\n\n2. **API குறி**: பதிவு செய்த பிறகு, நீங்கள் ஒரு API குறியை பெறுவீர்கள். இந்த குறி உங்கள் அனுமதியை உறுதிப்படுத்தும்.\n\n3. **API ஆவணங்கள்**: அடுத்து, நீங்கள் API ஆவணங்களை படிக்க வேண்டும். இந்த ஆவணங்களில் API என்ன செய்யும், எப்படி பயன்படுத்த வேண்டும் என்று விளக்கப்பட்டிருக்கும்.\n\n4. **API அழைப்பு**: பின்னர், நீங்கள் உங்கள் திட்டத்தில் API அழைப்பை செயல்படுத்த வேண்டும். இதற்கு நீங்கள் ஒரு நிரலாக்க மொழியை பயன்படுத்த வேண்டும்.\n\n5. **தரவு பரிமாற்றம்**: API அழைப்பு செய்த பிறகு, நீங்கள் தரவுகளை பரிமாற்ற வேண்டும். இதற்கு நீங்கள் JSON அல்லது XML போன்ற ஒரு தரவு வடிவத்தை பயன்படுத்த வேண்டும்.\n\n6. **பதில் செயலாக்கம்**: இறுதியாக, நீங்கள் API பதிலை செயலாக்க வேண்டும். இதற்கு நீங்கள் பதில் தரவுகளை பயன்படுத்த வேண்டும்.\n\nஇந்த
```

படிகளை பின்பற்றுவதன் மூலம், நீங்கள் API-ஐ வெற்றிகரமாக பயன்படுத்த முடியும். அதிக தகவலுக்கு, தயவுசெய்து எங்களுடன் தொடர்பு கொள்ளுங்கள்."

```
    },
    "logprobs": null,
    "finish_reason": "stop"
  }
],
"usage": {
  "queue_time": 0.051281635,
  "prompt_tokens": 133,
  "prompt_time": 0.006970025,
  "completion_tokens": 1349,
  "completion_time": 4.178998339,
  "total_tokens": 1482,
  "total_time": 4.185968364
},
"usage_breakdown": null,
"system_fingerprint": "fp_dae98b5ecb",
"x_groq": {
  "id": "req_01ky416ym0fsvbw40hmdnsfape",
  "seed": 773435217
},
"service_tier": "on_demand"
}
```

POST https://api.groq.com/openai/v1/chat/completions

Send

Docs Params Authorization Headers 9 Body Scripts Settings

Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

Schema Beautify

```
1 {
2   "model": "llama-3.3-70b-versatile",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are an API Specialist. Always respond in the same language as the user."
7     },
8     {
9       "role": "user",
10      "content": "மிகவும் நன்றி! இந்த API-ஐ எப்படி பயன்படுத்துவது என்று சொல்லங்கள்."
11    }
12  ]
13 }
```

Body Cookies 1 Headers 20 Test Results

200 OK • 4.32 s • 1.74 KB

Save Response

{ } JSON Preview Visualize

```
1 {
2   "id": "chatcmpl-d83e6bd2-9e4f-41b4-b54e-1ca0ea272586",
3   "object": "chat.completion",
4   "created": 1784694409,
5   "model": "llama-3.3-70b-versatile",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "நன்றி! இந்த API-ஐ பயன்படுத்துவதற்கு, நீங்கள் பின்வரும் படிகளை பின்பற்ற வேண்டும்:\n\n1. **API பதிவு**: முதலில், நீங்கள் அந்த API சேவையை வ  
இணையதளத்தில் பதிவு செய்ய வேண்டும். இதற்கு ஒரு பயனர் பெயர், மின்னஞ்சல் முகவரி மற்றும் கடவுச்சொல் தேவைப்படும்.\n2. **API குறி**: பதிவு செய்த  
நீங்கள் ஒரு API குறியை பெறுவீர்கள். இந்த குறி உங்கள் அனுமதியை உறுதிப்படுத்தும்.\n3. **API ஆவணங்கள்**: அடுத்து, நீங்கள் API ஆவணங்களை படிக்க  
இந்த ஆவணங்களில் API என்ன செய்யும், எப்படி பயன்படுத்த வேண்டும் என்று விளக்கப்பட்டிருக்கும்.\n4. **API அழைப்பு**: பின்னர், நீங்கள் உங்கள் திட்டத்தி  
API அழைப்பை செயல்படுத்த வேண்டும். இதற்கு நீங்கள் 350N அல்லது XML போன்ற ஒரு தரவு வடிவத்தை பயன்படுத்த வேண்டும்.\n5. **தரவு பரிமாற்றம்**: API அழைப்பு செய்த பிற  
தரவுகளை பரிமாற்ற வேண்டும். இதற்கு நீங்கள் JSON அல்லது XML போன்ற ஒரு தரவு வடிவத்தை பயன்படுத்த வேண்டும்.\n6. **பதில் செயலாக்கம்**: இறுதியாக  
API பதிலை செயலாக்க வேண்டும். இதற்கு நீங்கள் பதில் தரவுகளை பயன்படுத்த வேண்டும்.\nஇந்த படிகளை பின்பற்றுவதன் மூலம், நீங்கள் API-ஐ வெற்றிகரமா  
முடியும். அதிக தகவலுக்கு, தயவுசெய்து எங்களுடன் தொடர்பு கொள்ளுங்கள்."
12      },
13      "logprobs": null,
14      "finish_reason": "stop"
15    }
16  ]
17 }
```

Assignment 03: GenerateAPITestcasesFromSwaggerCode

Request Payload:

```
{
  "model": "openai/gpt-oss-120b",
  "messages": [
    {
      "role": "system",
      "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
    },
    {
      "role": "user",
      "content": "I - INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC - CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP - PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO - OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n- Add a new pet\n- Find pet by ID\n- Update an existing pet\n- Delete a pet"
    }
  ]
}
```

Response:

```
{
  "id": "chatcmpl-c83e461f-cd01-4f20-81ab-9d1e2b284cd1",
  "object": "chat.completion",
  "created": 1784694426,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
```

```

        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport
io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport
org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport
org.testng.annotations.Test;\n\nimport java.util.HashMap;\nimport
java.util.Map;\nimport java.util.Random;\n\npublic class PetstorePetCrudTest {\n\n
private static long petId;\n\n    @BeforeClass\n    public void setUp() {\n
RestAssured.baseURI = \"https://petstore.swagger.io/v2\";\n    }\n\n    @Test\n
public void createPet() {\n        petId = new Random().nextLong() & Long.MAX_VALUE;
// ensure positive\n        Map<String, Object> pet = new HashMap<>();\n
pet.put(\"id\", petId);\n        pet.put(\"name\", \"Fluffy\");\n
pet.put(\"status\", \"available\");\n\n        Response response =
RestAssured.given()\n            .contentType(ContentType.JSON)\n
            .body(pet)\n            .when()\n            .post(\"/pet\")\n
            .then()\n            .extract()\n            .response();\n\n
Assert.assertEquals(response.getStatusCode(), 200, \"Create pet status code\");\n
Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId, \"Created pet ID\");\n
Assert.assertEquals(response.jsonPath().getString(\"name\"), \"Fluffy\", \"Created pet
name\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\", \"Created pet status\");\n    }\n\n    @Test(dependsOnMethods =
\"createPet\")\n    public void getPetById() {\n        Response response =
RestAssured.given()\n            .accept(ContentType.JSON)\n
            .when()\n            .get(\"/pet/\" + petId)\n            .then()\n
            .extract()\n            .response();\n\n
Assert.assertEquals(response.getStatusCode(), 200, \"Get pet status code\");\n
Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId, \"Fetched pet ID\");\n
Assert.assertEquals(response.jsonPath().getString(\"name\"), \"Fluffy\", \"Fetched pet
name\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"),
\"available\", \"Fetched pet status\");\n    }\n\n    @Test(dependsOnMethods =
\"getPetById\")\n    public void updatePet() {\n        Map<String, Object> updatedPet
= new HashMap<>();\n        updatedPet.put(\"id\", petId);\n
updatedPet.put(\"name\", \"FluffyUpdated\");\n        updatedPet.put(\"status\",
\"sold\");\n\n        Response response = RestAssured.given()\n
            .contentType(ContentType.JSON)\n            .body(updatedPet)\n
            .when()\n            .put(\"/pet\")\n            .then()\n
            .extract()\n            .response();\n\n
Assert.assertEquals(response.getStatusCode(), 200, \"Update pet status code\");\n
Assert.assertEquals(response.jsonPath().getLong(\"id\"), petId, \"Updated pet ID\");\n
Assert.assertEquals(response.jsonPath().getString(\"name\"), \"FluffyUpdated\",
\"Updated pet name\");\n
Assert.assertEquals(response.jsonPath().getString(\"status\"), \"sold\", \"Updated pet
status\");\n    }\n\n    @Test(dependsOnMethods = \"updatePet\")\n    public void

```

```

DeletePet() {\n
    Response response = RestAssured.given()\n
.when()\n
.delete(\"/pet/\" + petId)\n
.then()\n
.extract()\n
.response();\n\n
Assert.assertEquals(response.getStatusCode(), 200, \"Delete pet status code\");\n
Assert.assertEquals(response.jsonPath().getLong(\"code\"), 200, \"Delete response\n
code\");\n
Assert.assertTrue(response.jsonPath().getString(\"message\").contains(String.valueOf(p\n
etId)), \"Delete response message\");\n    }\n\n",

    "reasoning": "We need to output Java code, no markdown fences, just\n
raw code. Must include imports, class, @BeforeClass, test methods for createPet,\n
getPetById, updatePet, deletePet. Use Rest Assured, TestNG, assertions. Use reusable\n
base URI. Use meaningful names. Include request body creation. Use assertions for\n
status code and important response fields. No placeholder comments. Let's craft\n
code.\n\nSwagger Petstore base URL is usually https://petstore.swagger.io/v2. Use\n
RestAssured.baseURI. Use static import of given().\n\nWill need to store petId for\n
later tests. Use a static long petId.\n\nImplement createPet test: POST /pet with JSON\n
body containing id, name, status. Use random id. Then assert status 200 and response\n
contains id, name.\n\ngetPetById test: GET /pet/{petId} assert 200 and\n
fields.\n\nupdatePet test: PUT /pet with changed name/status. assert 200.\n\ndeletePet\n
test: DELETE /pet/{petId} assert 200.\n\nUse TestNG @Test with priority to run in\n
order? Could use dependsOnMethods.\n\nLet's write code.\n\nAdd imports:\n
io.restassured.RestAssured, io.restassured.http.ContentType,\n
io.restassured.response.Response, org.testng.annotations.BeforeClass,\n
org.testng.annotations.Test, org.testng.Assert.\n\nWill use JsonObject building via\n
HashMap or using org.json. Simpler: use a Map<String,Object> for body and let\n
RestAssured serialize.\n\nLet's code."

    },
    "logprobs": null,
    "finish_reason": "stop"
}
],
"usage": {
    "queue_time": 0.158630139,
    "prompt_tokens": 396,
    "prompt_time": 0.018684069,
    "completion_tokens": 1096,
    "completion_time": 2.300786336,
    "total_tokens": 1492,
    "total_time": 2.319470405,
    "completion_tokens_details": {
        "reasoning_tokens": 313
    }
}

```

```

    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_84bf0227ec",
    "x_groq": {
      "id": "req_01ky417gnhftprfhak0j9ct1qj",
      "seed": 336920525
    },
    "service_tier": "on_demand"
  }
}

```

GenAI-Jul-HomeAssignment > S03-GenerateAPITestcasesFromSwaggerCode Share

POST ▼ <https://api.groq.com/openai/v1/chat/completions> Send

Docs Params Authorization Headers 9 **Body** ▼ Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL ☒ JSON ▼ Schema Beautify

```

1 {
2   "model": "openai/gpt-oss-128b",
3   "messages": [
4     {
5       "role": "system",
6       "content": "You are a Senior API Test Automation Architect with strong expertise in Java, Rest Assured, TestNG, Maven, and API schema-driven test generation."
7     },
8     {
9       "role": "user",
10      "content": "I - INSTRUCTION\nGenerate Rest Assured Java code for Swagger Petstore APIs.\n\nMANDATORY RULES:\n1) Generate ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nCONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nPERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nOUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- Use business-readable method names\n\nTARGET API:\nSwagger Petstore\nResource: Pet\nOperations required:\n"
11    ]
12  }

```

Body Cookies 1 Headers 20 Test Results 🔍 200 OK • 2.52 s • 2.34 KB 🔗 📄 🔍 🔗 🔗 Save Response ...

JSON ▼ Preview Visualize ▼

```

1 {
2   "id": "chatompl-c83e461f-cd91-4f28-81ab-9d1e2b284cd1",
3   "object": "chat.completion",
4   "created": 1784694426,
5   "model": "openai/gpt-oss-128b",
6   "choices": [
7     {
8       "index": 0,
9       "message": {
10        "role": "assistant",
11        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport java.util.HashMap;\nimport java.util.Map;\nimport java.util.Random;\n\npublic class PetstorePetCrudTest {\n\n    private static long petId;\n\n    @BeforeClass\n    public void setUp() {\n        RestAssured.baseURI = \"https://petstore.swagger.io/v2\";\n    }\n\n    @Test\n    public void createPet() {\n        petId = new Random().nextLong() & Long.MAX_VALUE; // ensure positive\n        Map<String, Object> pet = new HashMap<>();\n        pet.put(\"id\", petId);\n        pet.put(\"name\", \"Fluffy\");\n        pet.put(\"status\", \"available\");\n\n        Response response = RestAssured.given()\n            .body(pet)\n            .when()\n                .post(\"/pet/\")\n            .then()\n                .extract()\n            .response();\n\n        Assert.assertEquals(response.getStatusCode(), 200, \"Create pet status code\");\n        Assert.assertEquals(response.jsonPath().getString(\"name\"), \"Fluffy\");\n        Assert.assertEquals(response.jsonPath().getString(\"status\"), \"available\");\n        Assert.assertEquals(response.jsonPath().getString(\"created pet status\"), \"Created pet status\");\n    }\n\n    @Test(dependsOnMethods = {\"createPet\"})\n    public void getPetById() {\n        Response response = RestAssured.given()\n            .accept

```